Amazon Developer Hackathon: Winning Project Strategy

Oct 11, 2026 · @Evan Mastro

Bottom line

Build **Countersign**: an agent that checks every carrier "Delivered" scan against your Ring camera, and builds a timestamped evidence packet when the two disagree. Enter it in the **Ring** track (primary) and the **Alexa+** track (secondary), plus the **AWS Builder** mini-challenge.

The organizers confirmed on the forum that one project can be judged in several tracks and win at most one track prize. One build therefore buys two tracks' judging.

Why this beats the alternatives:

- **It is verification, not narration.** Ring already ships AI Video Descriptions, and Prime Video already uses AI to speed up audio description. "Describe what the camera sees" is an existing Amazon feature. Checking a carrier's claim against independent camera evidence is not (no direct competitor found in a quick search; treat as a hypothesis).
- **It matches the judges' own creative lists.** Ring's rubric names package/delivery management and event-based triggers. Alexa+'s names cross-service agentic workflows, state across sessions and rich cards.
- **Every dependency is free and open today.** Ring Developer Playground (simulated Package, Vehicle and Motion live-view streams, no device or subscription), EasyPost test-mode trackers, Bedrock, Strands, AgentCore. No Mac, TV or wearable required.
- **It fits your agent-evaluation background.** The core idea is an agent that refuses to trust a tool's output until a second source confirms it.

Fallback: Aside, an audio-description layer for Fire TV, if the Ring live-view spike fails the Day 2 gate.

Clock: submissions close Oct 23, 2026 at 12:00 PM PT (1:00 PM in Denver). That is twelve full days counting today, plus the morning of the 23rd. AWS promotional credits ran out on Oct 7, so Bedrock usage comes out of your own account.

Verified rules and facts

Everything below comes from the official Devpost pages, organizer forum replies, or official API docs, opened on Oct 11. Where a point is unverified, it sits in the open-questions list.

Topic	Verified fact	Source
Deadline	Submissions close Oct 23, 2026, 12:00 PM PT. Judging Nov 9–20; winners around Dec 3.	Rules
Field size	About 53,000 registered participants. Submission count unknown.	Overview
Judging	Stage 1 is pass/fail on theme fit and use of the required APIs. Stage 2 scores four equally weighted criteria: Tech Implementation, Design, Potential Impact, Quality of the Idea.	Rules
Obvious vs creative	The rules list obvious and creative examples per track. Ring creative: non-security uses, caretaking, package/delivery management, business integration, event triggers. Alexa+ creative: cross-service agentic workflows, state across sessions, purchasing, cards, MCP Apps.	Rules
Multiple tracks	One project can be judged in several tracks and win only one track prize, plus one mini-challenge prize.	Forum reply, Rules
AWS Builder	Any primary-track project using AWS services with documented integrations. Kiro Crew qualifies alone. One winner: $5,000 cash + $5,000 AWS credits. Describe AWS usage in the product feedback answer.	Rules

Topic	Verified fact	Source
Friction log	Optional. Each entry: task, steps, expected vs actual, severity, workaround, actionable suggestion. Amazon's team rates entries in Stage 1 and passes up to a 10% bonus to the Stage 2 panel.	Rules
Product feedback	Required for every tool, API and SDK used: what worked, what needs work, onboarding, would you build again.	Rules
Repository	GitHub only. Public with a license visible in the About panel, or private and shared with the listed reviewers (invites expire after 7 days). Alexa+, Bee and Ring repos must import and call the track technology at runtime.	Rules, FAQ
Video	Under 3 minutes, public on YouTube or Vimeo, English, showing the project running on its platform. No third-party trademarks or copyrighted media without permission.	Rules
Alexa+ access	The Alexa+ MCP Toolkit, CLI and web simulator are partner-only. Build a self-hosted MCP server (spec 2025-11-25+, Streamable HTTP) and demo it from your own web page acting as an MCP client.	FAQ
Alexa+ guidance	Ideas that are a latent LLM capability or an existing Alexa+ feature "would likely score poorly on Design and Quality of the Idea." Consumer use cases fit best; multimodal is encouraged.	Forum reply
Ring simulator	The Ring Developer Playground is the sanctioned simulator. It offers three simulated live-view streams (Package, Vehicle, Motion) through the API. Your own footage is allowed if labeled.	Forum reply, Release notes

Topic	Verified fact	Source
Ring API	Server-to-server only (CORS blocks browsers). No official SDK. Playground tokens last about 30 minutes. All media carries a mandatory watermark. Real-device staging needs a Ring Protection plan and US devices.	API docs, Develop, helloworld
Bee	Needs real data from a Bee device or an Apple Watch running Bee. Developer Mode appears only after pairing a Bee device; the docs list the iOS app as a prerequisite.	Rules, FAQ, Bee docs
Fire TV	An emulator or the Vega Virtual Device is acceptable. The Vega SDK supports macOS or Ubuntu only; Windows and WSL are not supported.	FAQ, Vega SDK
AWS credits	The $150 credit program ran out on Oct 7. One entrant reports a new AWS account with every Bedrock quota at 0.	Resources, Forum

Open questions to settle in the first 48 hours:

- Can a non-browser WebRTC client (Python aiortc) consume a Playground live-view session? The sample app does it from a browser; server-side is unverified.
- How the Playground exposes its three streams: as three devices, or as a parameter on one. Determines the demo scenario wiring.
- Whether Playground tokens can read Event History or image snapshots for simulated devices.
- Whether a multi-track entry gets a separate Stage 1 check per track. Assume both tracks' requirements must be met.

Phase one: track selection

Ring scores highest at 7.00, ahead of Alexa+ (6.85), Fire TV (6.45) and Bee (4.85). Competitor density per track is unpublished, so every Winning Potential score is a hypothesis built from barriers to entry.

Weights. Feasibility 20%, Integration Risk 20% and Winning Potential 20% dominate because a solo builder with twelve days loses more to a blocked dependency than to a weaker idea. Demo, Tech Depth and Commercial take 10% each. AWS Fit and Friction take 5% each because every track can earn them roughly equally.

Criterion (weight)	Fire TV	Alexa+	Bee	Ring
Winning Potential (20%)	6 · Three prizes up to $25K; real-platform build raises the bar, but AI-viewing ideas are likely crowded	4 · Lowest barrier (a web page and an MCP server) suggests the most entries; organizer warns latent-LLM ideas score low	7 · Hardware gate suggests few entries; Bee co-founders judge	7 · New API and server-side WebRTC thin the field; only two prizes
Solo Feasibility (20%)	6 · TV focus management and an emulator; Vega needs macOS or Ubuntu	9 · MCP server plus web client is squarely your stack	3 · Needs a device, the iOS app, pairing and days of real data first	7 · Playground token in minutes; frame capture is the hard part
Demo Impact (10%)	8 · A TV screen reads instantly	6 · A web-simulated assistant looks like every chat demo	6 · The data is invisible without a strong UI	7 · Real camera frames and a visible decision
Technical Depth (10%)	7 · Device platform plus backend AI	6 · MCP spec compliance is table stakes	6 · CLI and MCP are read-mostly wrappers	8 · OAuth, WHEP WebRTC, JSON:API, watermarked media

Criterion (weight)	Fire TV	Alexa+	Bee	Ring
AWS Fit (5%)	7 · Bedrock vision fits; AgentCore less so	8 · MCP, Strands and AgentCore fit naturally	6 · Bedrock over transcripts is the obvious use	8 · Ring's own CV guide prescribes Bedrock, Rekognition, SageMaker
Friction Opportunity (5%)	8 · Young Vega SDK, OS limits, virtual device	5 · Mature MCP SDKs; friction mostly on Bedrock	7 · Developer Mode gating, shared app_id	8 · Six API or doc changes between Sept 11 and Oct 9; Playground limits
Commercial Potential (10%)	6 · Consumer TV apps monetize poorly; publisher B2B possible	6 · No distribution path today	6 · Small installed base	7 · Ring Appstore supports paid apps and subscriptions
Integration Risk, higher = safer (20%)	6 · OS dependency and emulator quirks	9 · Nothing gated that you need	2 · Hardware, shipping and data accumulation	6 · Frame capture from simulated streams is unverified
Weighted total	**6.45**	**6.85**	**4.85**	**7.00**

Three different answers, depending on the question:

- **Highest potential prize:** Fire TV or Alexa+, each $25,000 cash plus $15,000 AWS credits for first.
- **Strongest risk-adjusted opportunity:** Ring. A thinner field (hypothesis), an API the judges built, and a sanctioned simulator.
- **Best return on development time:** one MCP-centric build entered in Ring and Alexa+. The Ring integration is the product; the MCP server is about a day of extra work and buys a second track's judging.

Recommendation: Ring as the primary track, with an Alexa+ co-entry. **Fallback track:** Fire TV.

What would change this:

- The Day 2 spike cannot pull frames from a Playground session by any route → switch to Fire TV.
- You already own a Bee and an iPhone with weeks of captured data → Bee's feasibility jumps from 3 to about 7, and it becomes a serious contender.
- An organizer says multi-track entries are judged on their weakest track → drop the Alexa+ co-entry; Ring stays primary.
- You work on a Mac and already know React Native for TV → Fire TV feasibility rises, though its concept problem (below) remains.

Phase two: concept screen

Sixteen concepts went in; four survived, one per track. Most cuts were for the same reason: Amazon already ships the feature, or the rules' own "creative" examples make the idea a magnet for other entrants.

Track	Concept	Verdict	Reason
Ring	**Countersign**: check carrier "Delivered" scans against the camera	Finalist	Reconciles two sources instead of describing one; package management is on Ring's creative list
Ring	AI door descriptions spoken to blind users	Cut	Ring Video Descriptions already writes AI text for motion alerts
Ring	Caregiver visit verification (aide arrived, stayed, left)	Cut	Another entrant's open-source emulator already ships home-aide and no-show scenarios; worker-monitoring privacy questions
Ring	Front-desk access control for small offices	Cut	Identifying people by face is a policy risk; Playground has no access hardware
Fire TV	**Aside**: on-demand and gap-timed audio description for blind and low-vision viewers	Finalist	Accessibility on the shared screen, spoken through Fire TV's own VoiceView

Track	Concept	Verdict	Reason
Fire TV	Computer-vision fitness coach	Cut	It is the rules' own creative example, so expect many; Peloton Guide and Tempo exist
Fire TV	AI sports second screen	Cut	Needs licensed live data and footage; video rights risk
Fire TV	Doorbell events that pause the show only when it matters	Cut	Fire TV already shows doorbell picture-in-picture, and Ring already classifies motion
Alexa+	**Fixwise**: appliance troubleshooting from your own manuals	Finalist	Stateful, multimodal, consumer; weakest on originality
Alexa+	Household logistics ledger	Cut	Alexa lists and reminders exist; organizer says existing features score low
Alexa+	Product recall radar	Cut	A forum thread shows another entrant building it
Alexa+	Family caregiving coordinator	Cut	Health data sensitivity and a crowded app category
Bee	**Loose Ends**: two-sided commitment tracking from conversations	Finalist	Uses Bee's real-time stream and todo write-back; conditional on owning hardware
Bee	Lecture capture to study decks	Cut	Your program is online, so there are no live lectures to capture
Bee	Spoken team discussions to GitHub issues	Cut	Close second, but weaker without a co-located team generating real data
Bee	Self-improvement coach on speaking habits	Cut	Hard to show value in under three minutes

The finalists are not equal. Countersign is the only one whose core move (refusing to trust a tool's claim until a second source agrees) is both novel and demonstrable with free, sanctioned tooling.

Finalist 1: Countersign (Ring + Alexa+)

A. Product concept

One-sentence pitch: When a carrier marks your package "Delivered," Countersign looks at your Ring camera within minutes, confirms or disputes the claim, and hands you a timestamped evidence packet if the two disagree.

Target user: Households with a Ring doorbell and frequent deliveries. Second segment: small businesses that receive stock at a street door (salons, clinics, small offices).

Pain point: A "Delivered" scan is a claim, not proof. Carriers sometimes scan early or at the wrong address, and merchants tell customers to go check their door-camera footage themselves. Doing that means scrubbing video to the right minute, saving clips and writing a claim, usually days later when the footage may be gone.

Existing alternatives:

Alternative	What it does	What it leaves undone
Ring Package Alerts and Video Descriptions	Detects or describes activity at the door	Never compares it with what the carrier claims
Carrier photo-on-delivery	The carrier's own evidence	The party being checked supplies the proof
Tracking apps (carrier apps, AfterShip)	Relay the carrier's status	Repeat the claim; do not test it
Driver-side proof-of-delivery AI (VerifyAI)	Checks the driver's photo for the shipper	Serves the sender, not the recipient

Countersign is the recipient's independent second witness, plus the reconciliation and paperwork.

Why now: The Ring Partner API reached general availability with WebRTC live view, and its Playground (May 2026) opened it to developers without hardware. Bedrock vision models can judge "is a parcel on this step" from a watermarked frame. MCP lets the same capability appear in Alexa+.

The killer moment: A split screen. The tracker on the left flips to *Delivered 3:02 PM*. Seconds later the right side opens the Ring live view, sees an empty step, and the card turns red: *Carrier says delivered. Your door says no.* One click opens the evidence packet,

with Ring's own watermark (device ID and timestamp) burned into every frame.

Business model: A paid Ring Appstore app (the platform supports paid apps and subscription webhooks), priced around $2–3 a month (hypothesis). Larger buyers: shipping-protection insurers and retailers, who pay per verified claim to cut fraudulent "never arrived" refunds. Expansion: early-scan detection, wrong-house deliveries, returns pickup verification, post-delivery theft re-checks.

B. Track and challenge architecture

Entry	Requirement (verified)	How Countersign meets it
Ring track (primary)	Use a Ring API, SDK or simulator at runtime; show it working through a simulator or device	Bearer-token calls to `api.amazonvision.com`: device discovery, configuration (encryption check), WHEP live-view sessions on Playground streams
Alexa+ track (secondary)	Self-hosted MCP server, spec 2025-11-25+, Streamable HTTP, shown in action	MCP server exposing `verify_delivery`, `get_delivery_ledger`, `build_claim_packet`, `recheck_door`; a web page acts as the MCP client with voice in and out
AWS Builder mini-challenge	AWS services with documented integrations; described in product feedback	Bedrock Converse (vision judgment), Strands Agents SDK (reconciliation agent), Bedrock AgentCore Runtime (hosts the agent); S3 and DynamoDB for evidence and ledger

Why the AWS pieces are essential, not decoration: the verdict needs visual judgment on unpredictable frames, which is where Ring's own computer vision guide places Bedrock vision models. The agent decides when to re-check, when to build a packet and how to answer a person's question. The verdict itself stays in deterministic code, so it is reproducible and auditable.

How judges see it: the dashboard's trace panel shows every Bedrock call with its model ID, request ID and latency; the README links the AgentCore deployment; the product feedback answer names each service and its job. Kiro Crew would qualify alone, but the

judges' creative bar is a multi-service pipeline, so use Kiro only if it genuinely speeds you up.

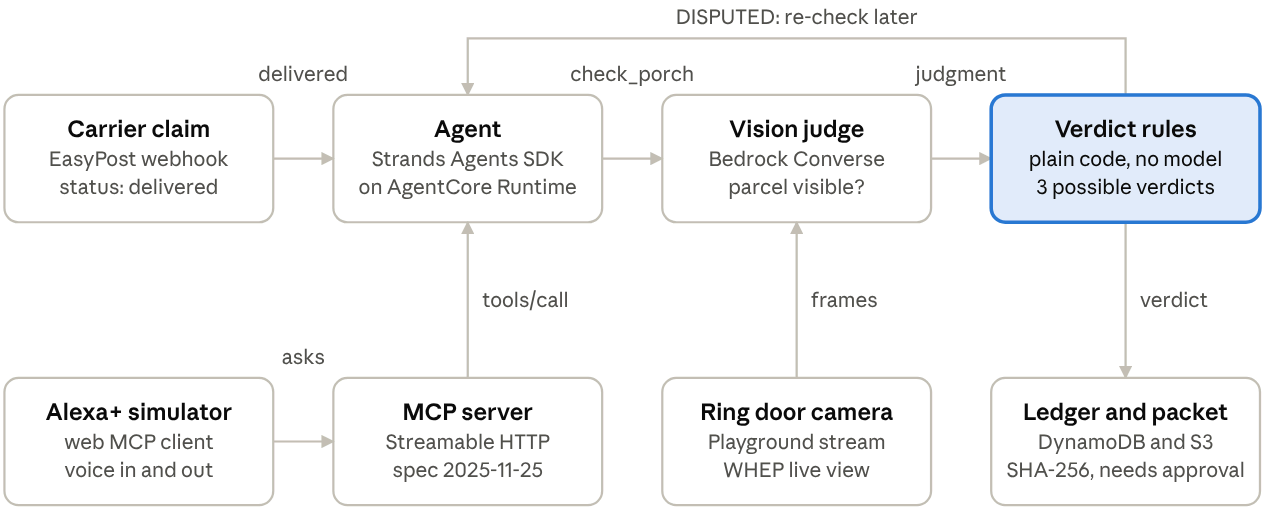
C. Core user workflow

1. **Trigger.** EasyPost posts a `tracker.updated` webhook with status `delivered` (test mode: tracking code `EZ4000000004`). The backend acknowledges at once and queues the job.
2. **Context.** The agent reads the tracker timeline (carrier, scan time), finds the door camera via `GET /v1/devices?include=status,configurations`, skips devices whose `e2e_encryption.enabled` is `true`, and opens a WHEP live-view session.
3. **Reasoning.** Three to five frames are sampled. Bedrock returns a structured judgment: parcel visible, confidence, where, the watermark time it read. A deterministic policy turns that into VERIFIED, DISPUTED or INCONCLUSIVE.
4. **Action.** VERIFIED writes a ledger entry and a quiet notice. DISPUTED schedules a re-check (10 minutes by default, 60 seconds in the demo, labeled on screen), then builds an evidence packet in S3 and drafts a claim that waits for your approval. Nothing is sent without it.
5. **Verification and communication.** The ledger shows the verdict with its frames and policy trace; the packet carries a SHA-256 hash; asking "did my package actually arrive?" through the MCP client returns the same verdict with an evidence card.

Failure path: A device that is offline (503), encrypted, or returning a low-confidence judgment yields INCONCLUSIVE with the reason in plain words: "I can't see the step clearly. Please check." An expired Ring token surfaces as an error to re-authenticate, never as a verdict. Watermark text never counts as evidence of a parcel.

D. Technical architecture

The model judges frames; plain rules issue the verdict



Countersign architecture · 8 components, 1 re-check loop

Two independent witnesses, the carrier's webhook and the Ring camera, meet at the verdict rules; the model only answers "is a parcel visible," and the Alexa+ simulator reaches the same agent through the MCP server.

- **Frontend:** Next.js dashboard with the ledger, the split-screen live view and the trace panel. A second page is the Alexa+ simulator: a real MCP client over Streamable HTTP using the official TypeScript SDK, with the browser's speech recognition and synthesis for voice.
- **Backend:** Python FastAPI service holding the EasyPost webhook receiver, Ring client, frame grabber, policy engine and the MCP server.
- **Frame capture:** aiortc as a WHEP client. Fallback ladder if it cannot negotiate: (1) headless Chromium running the official helloworld page's WebRTC code and capturing canvas frames; (2) a visible browser bridge tab doing the same, disclosed in the README.
- **AI:** Bedrock Converse with an image-capable model (a Claude Sonnet-class model or Amazon Nova Lite/Pro; confirm image input on the model card).
- **Agent:** a Strands agent with five tools (`get_tracker`, `check_porch`, `schedule_recheck`, `build_packet`, `get_ledger`), packaged for AgentCore Runtime.
- **Storage:** DynamoDB for the ledger; S3 for frames and packets.
- **Auth:** Playground bearer token and EasyPost test key from environment variables; webhook signature check; MCP endpoint behind a bearer token with Origin validation.

- **Deployment:** Docker Compose for judges to run locally; the agent also deployed to AgentCore Runtime; a Cloudflare tunnel for EasyPost webhooks during development.
- **Observability:** AgentCore's OpenTelemetry tracing plus the app's own trace table, which the UI renders.
- **Testing:** pytest against ring-sandbox, an MIT-licensed offline Ring API emulator, so tests never burn Playground tokens; a labeled frame set (at least 40 frames from the three streams plus your own porch photos) that reports precision and recall for the judge.

Riskiest integration: getting frames out of a Playground live-view session from a server. **Simplest fallback:** the browser route the official sample already proves works.

E. Implementation blueprint

Interfaces marked **verified** were read in official docs or repos on Oct 11. Anything marked **check** must be confirmed on Day 0–1 before you build on it.

```
countersign/
  README.md            one-command setup, demo scenarios, simulator
disclosure
  LICENSE              Apache-2.0 (shows in GitHub's About panel)
  docker-compose.yml   backend + web + local DynamoDB for judges
  .env.example
  backend/
    pyproject.toml
    countersign/
      main.py          FastAPI: /webhooks/easypost, /api/ledger, /api/trace
(SSE)
      ring_client.py   Amazon Vision API calls
      frames.py        WHEP session via aiortc -> JPEG frames
      judge.py         Bedrock Converse vision judgment -> Judgment model
      policy.py        deterministic VERIFIED / DISPUTED / INCONCLUSIVE
      agent.py         Strands agent + tools
      agentcore_app.py AgentCore Runtime entrypoint
      mcp_server.py    MCP tools over Streamable HTTP
      packets.py       evidence packet to S3 + SHA-256
      scenarios.py     simulator scenarios (EasyPost test tracker +
Playground stream)
    tests/             pytest with ring-sandbox emulator fixtures
  eval/
```

```
    frames/ labels.csv run_eval.py   precision / recall of the judge
  web/                Next.js dashboard + Alexa+ simulator page (MCP client)
  docs/
    FRICTION.md  DEVLOG.md  PRODUCT_FEEDBACK.md  architecture.svg
```

Dependencies (`backend/pyproject.toml`): `fastapi` , `uvicorn` , `httpx` , `aiortc` , `pillow` , `boto3` , `strands-agents` , `strands-agents-tools` , `bedrock-agentcore` , `mcp` , `easypost` , `pydantic` ; dev: `pytest` , `ring-sandbox[server]` . Web: `next` , `@modelcontextprotocol/sdk` .

Environment variables: `RING_ACCESS_TOKEN` (Playground, about 30 minutes), `RING_DOOR_DEVICE_ID` , `EASYPOST_API_KEY` (test key), `EASYPOST_WEBHOOK_SECRET` , `AWS_REGION` , `BEDROCK_MODEL_ID` , `LEDGER_TABLE` , `EVIDENCE_BUCKET` , `MCP_BEARER_TOKEN` , `RECHECK_DELAY_S` (600 default, 60 in demo).

Ring client — endpoints, methods and content types **verified** in the API reference. Where the WHEP answer and session URL come back is **check** (standard WHEP returns the answer SDP in the body and the session in a `Location` header).

```
AVA = "https://api.amazonvision.com"

async def list_devices(http, token):
    r = await http.get(f"{AVA}/v1/devices",
                       params={"include":
"status,capabilities,configurations"},
                       headers={"Authorization": f"Bearer {token}"})
    r.raise_for_status()
    return r.json()  # JSON:API document

async def start_whep(http, token, device_id, offer_sdp):
    r = await http.post(f"
{AVA}/v1/devices/{device_id}/media/streaming/whep/sessions",
                        content=offer_sdp,
                        headers={"Authorization": f"Bearer {token}",
                                 "Content-Type": "application/sdp"})
    r.raise_for_status()
    return r.text, r.headers.get("Location")  # check on Day 1

async def stop_whep(http, token, device_id, session_id):
    await http.delete(f"
{AVA}/v1/devices/{device_id}/media/streaming/whep/sessions/{session_id}",
                      headers={"Authorization": f"Bearer {token}"})
```

Frame grabber — aiortc's API is standard; using it against Ring's WHEP endpoint is the **Day 1 spike**. aiortc gathers ICE candidates before the offer is final, which suits WHEP's single POST.

```
from aiortc import RTCPeerConnection, RTCSessionDescription

async def grab_frames(http, token, device_id, n=4):
    pc = RTCPeerConnection()
    pc.addTransceiver("video", direction="recvonly")
    await pc.setLocalDescription(await pc.createOffer())
    answer, session = await start_whep(http, token, device_id,
pc.localDescription.sdp)
    frames = []
    @pc.on("track")
    async def on_track(track):
        while len(frames) < n:
            frames.append((await track.recv()).to_image())  # PIL image,
watermark included
    await pc.setRemoteDescription(RTCSessionDescription(sdp=answer,
type="answer"))
    ...  # wait with timeout, then stop_whep + pc.close()
```

Judge — Bedrock Converse with an image content block **verified** as the boto3 `bedrock-runtime` shape; model ID **check** for image input in your region.

```
resp = bedrock.converse(
    modelId=MODEL_ID,
    system=[{"text": JUDGE_RULES}],  # describe parcels only; ignore the Ring
watermark; never identify people
    messages=[{"role": "user", "content": [
        {"image": {"format": "jpeg", "source": {"bytes": jpeg}}},
        {"text": "Return JSON: parcel_visible, confidence, location,
watermark_time, notes"}]}],
    inferenceConfig={"maxTokens": 300, "temperature": 0})
```

Policy — plain code, so a verdict is reproducible without a model:

```
def decide(j: Judgment, scan_at: datetime, seen_at: datetime) -> Verdict:
    if j.unreadable:                          return Verdict("INCONCLUSIVE",
j.reason)
    if j.parcel_visible and j.confidence >= 0.8:      return
Verdict("VERIFIED")
    if not j.parcel_visible and j.confidence >= 0.8 and seen_at >= scan_at:
                                              return Verdict("DISPUTED", "no
parcel after scan")
    return Verdict("INCONCLUSIVE", "low confidence")
```

Agent — `from strands import Agent` and `pip install strands-agents strands-agents-tools` **verified** in the Strands repo. The `tool` decorator export and `strands.models.BedrockModel` path are **check**.

```
from strands import Agent, tool

@tool
def check_porch(tracking_code: str) -> dict:
    """Open a Ring live view on the door camera, sample frames, return the
policy verdict."""

agent = Agent(tools=[get_tracker, check_porch, schedule_recheck,
build_packet, get_ledger],
              system_prompt=AGENT_RULES)  # the agent picks steps;
policy.decide picks verdicts
```

AgentCore entrypoint — **verified** in the AgentCore SDK README, deployed with `npm i -g @aws/agentcore`, `agentcore create --name Countersign --defaults`, `agentcore deploy`.

```
from bedrock_agentcore import BedrockAgentCoreApp
app = BedrockAgentCoreApp()

@app.entrypoint
async def handler(request):
    prompt = request.get("prompt")
    if not isinstance(prompt, str):
        raise ValueError("prompt must be a string")
    async for event in agent.stream_async(prompt):
        yield event

app.run()
```

MCP server — the Python SDK's `FastMCP` with `transport="streamable-http"` is the **check** item: confirm the negotiated `protocolVersion` is 2025-11-25 or later. If it lags, move only this layer to the TypeScript SDK, which another entrant reports [negotiating 2025-11-25](#).

```
from mcp.server.fastmcp import FastMCP
mcp = FastMCP("countersign")

@mcp.tool()
async def verify_delivery(tracking_code: str) -> dict:
    """Has this package actually arrived? Checks the carrier claim against
the Ring camera."""

mcp.run(transport="streamable-http")
```

MCP tool	Input	Output
`verify_delivery`	`tracking_code` (string), `carrier` (optional)	`status` (VERIFIED, DISPUTED, INCONCLUSIVE, PENDING), `carrier_scan_at`, `checked_at`, `confidence`, `evidence[]` (frame URL, watermark time), `reason`
`recheck_door`	`tracking_code`	same shape as `verify_delivery`
`get_delivery_ledger`	`since` (ISO date, optional)	list of ledger entries

MCP tool	Input	Output
`build_claim_packet`	`tracking_code`	`packet_url`, `sha256`, `draft_text`, `requires_approval: true`

EasyPost test trackers — test codes such as `EZ4000000004` (delivered) and `tracker.updated` webhook events are **verified** in the tracking guide; test mode sends a single event in a supported status. The Python client call shape is **check**.

Simulator entry point: `python -m countersign.scenarios delivered_seen | delivered_not_seen | unreadable`. Each creates a real test-mode tracker and binds its address to one Playground stream (Package, Motion, or a forced timeout). The README and video say exactly that: the scenario wiring is staged, the API calls and the judgment are live.

F. Three-minute demo script

Record each segment with a fresh Playground token, and keep a "Ring Developer Playground: simulated streams" label visible whenever a stream is on screen. Blank out carrier logos and brand names: the rules bar third-party trademarks in the video without permission.

Time	On screen	Narration	Evidence judges should see
0:00–0:20 Hook	A phone notification reading "Delivered 3:02 PM," then your own empty porch (labeled as your footage)	"Carriers sometimes mark a package delivered before it is. When they do, proving it is your job. Countersign makes your doorbell the second witness."	Title card: Ring track, Alexa+ track, AWS Builder

Time	On screen	Narration	Evidence judges should see
0:20–0:50 Trigger	Dashboard split screen: EasyPost tracker on the left with its TEST MODE badge, Ring live-view panel on the right. You create a tracker; the webhook lands; the tracker flips to Delivered	"A carrier scan just said delivered. Watch what happens without me touching anything."	Raw webhook JSON in the trace drawer; test-mode label visible
0:50–1:50 Intelligence	Trace panel streams step by step: device discovery → encryption check → WHEP session created → four sampled frames with Ring's watermark → Bedrock call (model ID, latency) → judgment JSON → the policy rule that fired. Case 1 on the Package stream ends VERIFIED. Case 2 on the Motion stream ends DISPUTED, re-check in 60 s	"The model only judges the picture. The verdict comes from rules you can read, so the same evidence always gives the same answer."	Real HTTP status codes; frames carrying Ring's device ID and timestamp; per-call Bedrock request IDs

Time	On screen	Narration	Evidence judges should see
1:50–2:30 Outcome	The re-check runs and stays empty. The evidence packet opens: frames, carrier timeline, policy trace, SHA-256, a drafted claim with an Approve button. Then Case 3: the door feed times out, and the card reads INCONCLUSIVE: "I can't see the step clearly. Please check." Then the Alexa+ simulator page: you ask aloud "Did my package actually arrive?"	"Nothing goes out without your approval, and when Countersign can't see, it says so instead of guessing."	The MCP client's `initialize`, `tools/list` and `tools/call` messages visible in a side log; negotiated protocol version 2025-11-25 shown; spoken answer plus the evidence card
2:30–3:00 Proof	AgentCore Runtime console with the deployed agent; one trace; the eval panel (precision and recall on your labeled frames, real measured numbers only); closing card	"Delivered is a claim. Countersign checks it."	Deployment ARN, OpenTelemetry trace, eval figures

If you run long, cut Case 3 to five seconds before cutting the Alexa+ beat: the Alexa+ judges need to see the MCP server working.

G. Friction log plan

Every item below is a **potential** friction point until it happens to you. Keep `docs/DEVLOG.md` from the first hour: a timestamp, the command, and the result for every integration attempt. That log is the raw material; the official entry format is filled from it, never from memory.

#	Tool	What to watch for	Why it could happen	How to reproduce and capture	Status today
1	Ring Playground + WHEP	A non-browser WebRTC client fails to negotiate (codec or profile mismatch, missing `Location` header, ICE needing TURN)	The official sample only shows the browser path	Minimal `whep_probe.py`; save offer and answer SDP, HTTP status and headers (token redacted), aiortc debug log, time to first frame	Plausible risk
2	Ring docs	Ring's MCP page says API credentials need the Developer Early Access Program; the hackathon resources say a free Ring account gives API access	Two docs written at different times	Screenshot both pages with URLs and dates; note what you actually needed	Verified existing inconsistency; log it only if it cost you time
3	Ring media + Bedrock	The vision model reads the mandatory watermark (logo, device ID, timestamp) as content, or misses parcels near the frame corners	The watermark is documented and cannot be removed	Run the eval set with and without a corner mask; record the precision and recall change	Plausible risk; log measured impact only
4	Ring Playground token	A scheduled re-check fails with 401 because the 30-minute token expired mid-flow	Documented lifetime, but it collides with delayed agent work	Capture the 401 response and the timeline; workaround: re-auth prompt plus a short demo delay	Documented behavior; log as a developer-experience suggestion only if it disrupts you

#	Tool	What to watch for	Why it could happen	How to reproduce and capture	Status today
5	Bedrock or the MCP Python SDK	Model quota at 0 on your account, or `FastMCP` negotiating a protocol version older than 2025-11-25	Another entrant reported zero quotas; SDK release timing is unknown	Save the error responses and request IDs, or the `initialize` result showing `protocolVersion`	Unknown until Day 0

What qualifies: the rules ask for the task attempted, the steps taken, expected versus actual, a severity rating, the workaround and an actionable suggestion. Entries about Ring, Alexa+/MCP and AWS tools all fit, since the product teams receive the feedback. Another entrant's reported issue is never yours to log.

Entry template for `docs/FRICTION.md`:

```
## FL-01: <one-line title>
- Tool / version:
- Task attempted:
- Steps taken: 1. … 2. … 3. …
- Expected:
- Actual: (paste status codes, error text, log excerpt; link the screenshot)
- Severity: blocker | major | minor
- Workaround:
- Suggestion:
- Evidence: DEVLOG timestamps, commit hash
```

Contingency if none of the targeted items occurs: log onboarding measurements honestly from the dev log, such as minutes from sign-up to first live-view frame, to first Bedrock vision answer, and to first AgentCore deploy, with the specific step that took longest and what would have shortened it. That is still "how onboarding felt," which the rules ask for, and it is true.

H. Twelve-day execution plan

The go/no-go gate is the end of Tuesday, Oct 13: frames from at least two Playground streams by any rung of the fallback ladder. No frames by then means switching to Aside with nine days left. Submission is planned for Friday morning, three hours before the deadline.

Day	Date	Build	Testable deliverable
0	Sun Oct 11	Accounts and smoke tests: Devpost, Ring Playground token, EasyPost test key, Bedrock vision call in your existing AWS account	`GET /v1/devices` returns 200; a test tracker exists; Bedrock answers a question about a JPEG
1	Mon Oct 12	WHEP spike with aiortc; helloworld browser path as baseline	One JPEG per Playground stream on disk, watermark visible, or captured errors
2	Tue Oct 13	Fallback ladder if needed; judge v0 on captured frames	**Gate:** frames from 2+ streams and judgment JSON for each. Fail → Aside
3	Wed Oct 14	Vertical slice: EasyPost webhook → frames → judge → policy → DynamoDB ledger	`delivered_seen` scenario runs end to end from the CLI
4	Thu Oct 15	DISPUTED path, scheduled re-check, S3 evidence packet with SHA-256, INCONCLUSIVE path	Three scenarios yield three different verdicts; pytest passes against ring-sandbox
5	Fri Oct 16	MCP server, four tools, Streamable HTTP	A script runs `initialize` → `tools/list` → `tools/call`; protocol version 2025-11-25 or later recorded
6	Sat Oct 17	Strands agent; deploy to AgentCore Runtime	The deployed agent answers "did tracker X arrive?" through an AgentCore invoke
7	Sun Oct 18	Next.js dashboard, trace panel, Alexa+ simulator page with voice	The full demo runs in a browser with no terminal visible
8	Mon Oct 19	Labeled frame set (40+), threshold tuning; MCP App evidence card if ahead	Eval report with precision and recall committed
9	Tue Oct 20	Reliability: token expiry, retries, offline and encrypted devices; README on a clean machine	Fresh clone to running demo in 10 minutes or less, timed

Day	Date	Build	Testable deliverable
10	Wed Oct 21	Friction log and product feedback from DEVLOG; record demo takes	Friction entries with evidence; rough cut under 3:00
11	Thu Oct 22	Final video, Devpost text, license in About panel, architecture image	Video public on YouTube; Devpost draft complete
12	Fri Oct 23	Final review and submit by 10:00 AM MT	Submission confirmed, three hours before the 1:00 PM MT close

Buffer rule: if Day 7 slips, cut the MCP App card and the voice input first. The ledger, the three verdicts, the packet and a working MCP `tools/call` are the submission; everything else is polish.

I. Judge objections and failure analysis

Question	Honest answer
Strongest argument against	Most packages arrive fine, so on most days the product has nothing to say. Answer: it runs on every delivery at no effort, and its value is the evidence on the day it matters. Lead the pitch with the insurer and retailer buyer, who sees every dispute.
Most obvious competing implementation	Ring's own package detection plus a manual check, or another Ring entrant doing "package detected" alerts. The reconciliation and the evidence chain are what those lack; make that the headline, never detection.
Weakest part of the architecture	Frames from Playground streams. Three fixed simulated streams also mean the demo cannot show a real theft or a late delivery arriving.
Most likely demo failure	A token expiring or a WHEP session failing mid-take. Keep a 30-second pre-flight script that refreshes the token and opens one session before every take.
Biggest reason it might look unoriginal	"Package detection is a Ring feature." Never say "detect"; say "verify the carrier's claim."

Question	Honest answer
Smallest change that strengthens it	Put one real measured number on screen, such as false disputes on the labeled frame set. A verifier that shows its own error rate is hard to dismiss.

Finalist 2: Aside (Fire TV)

The strongest Fire TV idea, and the fallback if the Ring spike fails. Its problem is originality against Amazon's own work, not feasibility.

A. Concept. *Pitch:* blind and low-vision viewers press one remote button to hear what is on screen, and between lines of dialogue hear what is happening, inside any publisher's Fire TV app. *Target:* blind and low-vision viewers; paying customer is the long tail of Fire TV app publishers without audio-description (AD) tracks. *Pain:* human AD costs $15 to $75 per minute, so most catalogs have none. *Alternatives:* Prime Video already uses AI to find dialogue gaps and has the largest AD library in the US; Verbit and 3Play sell AI AD to publishers. Aside's difference is interactive, on-demand description inside the app, spoken through VoiceView. *Why now:* vision models on Bedrock; Vega's `announceForAccessibility` lets an app speak through VoiceView. *Killer moment:* the screen goes dark to simulate no sight; a button press says "A woman in a red coat climbs onto the train roof as it pulls away." *Business:* a per-minute SDK for publishers.

B. Tracks. Fire TV primary (runs on the Vega Virtual Device or a Fire OS emulator). AWS Builder: Bedrock vision, Amazon Transcribe for dialogue gaps, Amazon Polly when VoiceView is off, S3. A Ring co-entry is weak because Ring Video Descriptions already exists.

C. Workflow. (1) Viewer presses Describe. (2) The app sends title and playback position; the backend extracts the frame with ffmpeg and the nearby subtitles. (3) Bedrock writes a description sized to the gap. (4) The app pauses, speaks through VoiceView or a Polly clip, resumes. (5) Low confidence → "I can't tell what's happening here," never a guess.

D. Architecture. The multi-TV React Native sample (Vega, Fire OS, web from one codebase); FastAPI backend; Bedrock; Polly; S3. Content: Blender Foundation open movies under CC-BY, which the FAQ says to confirm with organizers. *Riskiest:* VoiceView behavior on the virtual device, and the Vega SDK's macOS/Ubuntu-only support. *Fallback:* Fire OS on the Android TV emulator with Polly audio.

E. Blueprint. `apps/vega` and `apps/expo-multi-tv` from the sample; `backend/describe.py` (ffmpeg frame + Bedrock Converse); `backend/gaps.py` (Transcribe word timestamps → silences); `shared-ui/DescribeButton.tsx` calling `AccessibilityInfo.announceForAccessibility`.

F. Demo. Hook on a dark screen; Describe on demand; an automatic description fills a dialogue gap; a low-confidence refusal; the AWS console. The payoff is heard rather than seen, which is risky for judges watching muted.

G. Friction. Vega SDK OS limits (verified, documented); VoiceView on the virtual device (unverified); Transcribe gap detection on music-heavy scenes (plausible).

H. Plan. Same gate structure, but Day 1 becomes the Vega SDK install and a VoiceView announcement test.

I. Objections. *Strongest against:* "Prime Video already does this at scale." *Obvious competitor:* any "ask about this scene" Fire TV entry. *Weakest part:* description quality; an RNIB study found AI AD fluent but often inaccurate. *Likely demo failure:* audio sync. *Unoriginal because:* image description is a latent model capability. *Smallest improvement:* recruit two blind testers and quote their reactions with permission.

Finalist 3: Fixwise (Alexa+)

The most feasible finalist and the least original. It survives as the Alexa+-only option, not as a contender.

A. Concept. *Pitch:* "Alexa, the dishwasher says E24" — Fixwise finds your exact model in your saved appliance registry, opens that model's manual at the fix, walks you through it step by step with diagram cards, and finds the part. *Target:* homeowners and renters. *Pain:* manuals are lost, error codes are model-specific, and generic answers are often wrong for your unit. *Alternatives:* manufacturer apps, YouTube, any chatbot with an uploaded PDF. *Why now:* MCP Apps let a voice answer carry a visual card. *Killer moment:* an error code becomes the right page, the right steps and the part number in one answer. *Business:* parts affiliate revenue and extended-warranty referrals.

B. Tracks. Alexa+ only. AWS Builder: Bedrock Knowledge Bases over manuals in S3, a Strands agent, AgentCore Memory for the registry.

C. Workflow. Error code spoken → registry lookup → retrieval from that model's manual → stepwise guide with a card per step → "did that clear it?" → parts search if not. Failure path: manual not on file → ask for a photo of the model label.

D–E. Architecture and blueprint. TypeScript MCP server (official SDK, Streamable HTTP) with `register_appliance`, `diagnose`, `next_step`, `find_part`; an MCP App card per step via `@modelcontextprotocol/ext-apps`; a web MCP client as the Alexa+ simulator. *Riskiest:* none worth naming, which says something about its ambition.

F. Demo. A real dishwasher manual and a real error code; the card sequence; a refusal when the model label is unreadable.

G. Friction. Thin: mature MCP SDKs. Bedrock Knowledge Base setup and quotas are the only likely sources.

H. Plan. Seven build days; the risk is boredom, not schedule.

I. Objections. *Strongest against:* retrieval over manuals is exactly the "latent capability of modern LLMs" the Alexa+ organizer said scores poorly on Design and Quality of the Idea. *Obvious competitor:* ChatGPT with the PDF attached. *Weakest part:* real purchasing is impossible in the sandbox. *Unoriginal because:* everyone has built document Q&A. *Smallest improvement:* none that fixes the originality gap.

Finalist 4: Loose Ends (Bee)

A good idea held back by access. If you already own a Bee and an iPhone with captured conversations, read this one twice; if not, the device, pairing and days of real data make it the riskiest finalist.

A. Concept. *Pitch:* Loose Ends catches the promises you make in conversation ("I'll send the deck Friday") and the ones made to you ("she'll get back to me Tuesday"), turns them into dated Bee todos, and nudges you before either side lets one lapse. *Target:* consultants, managers, anyone whose job is follow-through. *Pain:* commitments made out loud are the ones nobody writes down. *Alternatives:* Bee's own todo suggestions (the MCP server exposes `bee_get_todo_suggestions`), and meeting tools that list action items. Loose Ends adds the "waiting on others" side and real-time capture outside meetings. *Why now:* `bee stream` delivers utterances as they are transcribed. *Killer moment:* you say "I'll send that over by Thursday" mid-conversation, and the todo appears in the Bee app seconds later with the date set. *Business:* a paid add-on for professionals.

B. Tracks. Bee primary. AWS Builder: Bedrock extracts commitments; Strands decides create, update or ignore.

C. Workflow. `bee stream --types new-utterance --json` feeds the extractor → Bedrock returns {who owes whom, what, when, confidence} → confident items become `bee_create_todo` calls; uncertain ones wait for confirmation → a daily digest of open commitments both ways. Failure path: no date stated → ask, never invent one.

D–E. Architecture and blueprint. The Bee CLI (verified commands: `bee login`, `bee stream`, `bee mcp serve`, `bee mcp serve-http`) as the data layer; a Python worker; a small web digest. All verified in the Bee docs.

F. Demo. A live conversation on camera, the todo appearing on the phone, the digest. Strong if the capture is reliable on the day.

G. Friction. Developer Mode appearing only after device pairing (an organizer FAQ), the shared app_id with no per-app credentials (FAQ), and the stream's documented at-most-once delivery.

H. Plan. Day 0–2 is hardware: order, pair, enable Developer Mode, capture real conversations. Nothing else can start until data exists.

I. Objections. *Strongest against:* Bee already suggests todos from conversations. *Obvious competitor:* meeting-notes tools. *Weakest part:* extraction accuracy on casual speech. *Likely demo failure:* transcription lag on camera. *Smallest improvement:* show a commitment someone else made to you resolving days later.

Phase four: ranking and winner

Countersign ranks first at 7.85, a full point ahead of Aside (6.65). Fixwise (5.75) and Loose Ends (5.70) trail; Loose Ends rises to 6.30 only if you already own a Bee.

Criterion (weight)	Countersign	Aside	Fixwise	Loose Ends
Winning Potential (25%)	8	6	4	6
Technical Differentiation (20%)	8	7	5	6
Demo Quality (15%)	8	8	6	7
Execution Feasibility (15%)	7	5	9	3 (7 with a Bee)
AWS Challenge Integration (10%)	8	8	7	6
Commercial Value (10%)	8	6	6	6
Friction Log Opportunity (5%)	8	8	4	6

Criterion (weight)	Countersign	Aside	Fixwise	Loose Ends
Weighted total	**7.85**	**6.65**	**5.75**	**5.70 (6.30)**

The differences that decide it:

- **Originality against Amazon's own products.** Aside, Fixwise and Loose Ends each overlap something Amazon or the platform already ships: Prime Video's AI-assisted audio description, the latent document Q&A the Alexa+ organizer warned about, Bee's own todo suggestions. Countersign overlaps none of them.
- **Feasibility without hardware or a specific OS.** Countersign needs nothing you don't have. Aside needs macOS or Ubuntu for Vega; Loose Ends needs a device and days of data.
- **Two tracks from one build.** Only Countersign earns a credible second track (Alexa+) at the cost of about a day.

The winner: Countersign

- **Why it beats the other three:** it is the only finalist whose core act, refusing to trust a claim until independent evidence agrees, is new, useful and visible on screen in seconds.
- **Why it is achievable:** four verified APIs (Ring Partner API, EasyPost, Bedrock, MCP) and one unverified step (server-side frames), with a browser fallback the official sample already proves.
- **Why it is hard to dismiss as a generic AI project:** the model only looks at pictures. The verdict comes from readable rules, every claim carries Ring's watermark as provenance, and the system says "I can't tell" when it can't.
- **What judges will remember:** "Carrier says delivered. Your door says no."
- **What could stop it winning:** a stronger Ring entry in a thin two-prize field; the Playground's three fixed streams making the demo feel staged; or Bedrock access on your account. Address the first with the eval number on screen, the second with explicit disclosure, the third on Day 0.

The fallback: Aside

Use it if the Day 2 gate fails, meaning no frames from Playground sessions by any route. It shares the Bedrock vision judge, so roughly a day of Countersign work carries over.

The cut list

- Real Ring webhooks and one-way account linking (need real devices, a Ring Protection plan and an HTTPS app registration)
- Actually sending claims to a carrier or merchant; the packet and draft stop at your approval
- Identifying people, faces or vehicles; descriptions cover parcels and the step only
- Post-delivery theft detection, wrong-house detection, multi-camera support
- Ring Chime audio playback (needs the separately granted Chime Controls capability)
- A Fire TV surface, a mobile app, payments or subscriptions
- Production EasyPost trackers in the demo (supported in code, kept out of the video)
- AgentCore Memory, the MCP App card and voice input, in that order, if the schedule slips

The first 24 hours

The goal by tomorrow evening is one Ring frame written to disk by code you control, or a captured error that tells you which fallback rung to climb.

1. **Start the dev log.** Create the public GitHub repo `countersign` with an Apache-2.0 license, plus `docs/DEVLOG.md` and `docs/FRICTION.md`. Timestamp every step below in the log as you go.
2. **Ring Playground.** Sign in to the Amazon developer portal, open the Ring Developer Playground, click Generate Token. Clone ring-api-helloworld and run `python scripts/list_devices.py --token "…"`. Record how the Package, Vehicle and Motion streams appear: as separate devices or otherwise.
3. **Prove WHEP works at all.** Run the helloworld web app with `RING_ACCESS_TOKEN` set and open a live view on each stream in the browser. This is the baseline the server-side spike must match.
4. **Bedrock.** In your existing AWS account (not a fresh one), confirm access and quotas for an image-capable model in `us-east-1` or `us-west-2`. Send one `converse` call with a screenshot from step 3 and the question "Is there a parcel on the step?" If quotas read 0, open a support case at once and log it.
5. **EasyPost.** Create an account, copy the test API key, create a tracker with code `EZ4000000004`, and point a webhook at a Cloudflare tunnel to a 10-line FastAPI endpoint. Confirm the `tracker.updated` event arrives.

6. **MCP version check.** Run a minimal `FastMCP` server with `transport="streamable-http"`, send `initialize`, and record the `protocolVersion` it returns. If it is older than 2025-11-25, plan the MCP layer in the TypeScript SDK.
7. **Start the frame spike.** Write `whep_probe.py`: aiortc offer → `POST …/media/streaming/whep/sessions` → answer → save four JPEGs. Save the SDPs, status codes and headers whatever happens.
8. **Ask one question on the hackathon forum:** whether a project entered in Ring and Alexa+ is checked against each track's Stage 1 requirements separately. The answer only changes how much video time the Alexa+ beat needs.

Project brief

Field	Answer
Product name	Countersign
Track	Ring (primary) and Alexa+ (secondary entry); AWS Builder mini-challenge
One-sentence pitch	When a carrier marks your package delivered, Countersign checks your Ring camera, confirms or disputes the claim, and builds the evidence packet if they disagree.
Core differentiator	Verifies a third party's claim with independent evidence, and the verdict comes from readable rules rather than the model
Architecture	EasyPost webhook → FastAPI → Ring WHEP live view → frames → Bedrock vision judgment → deterministic policy → DynamoDB ledger and S3 packet; Strands agent on AgentCore Runtime; MCP server over Streamable HTTP; Next.js dashboard and Alexa+ simulator page
Essential integrations	Ring Partner API via the Developer Playground; EasyPost test-mode trackers; Amazon Bedrock Converse; Strands Agents SDK; Bedrock AgentCore Runtime; MCP 2025-11-25

Field	Answer
MVP scope	Three verdicts (verified, disputed, inconclusive), a scheduled re-check, an evidence packet awaiting approval, four MCP tools, a dashboard with a live trace, an eval report on labeled frames
Biggest risk	Pulling frames from a Playground live-view session in server code; the gate is end of day Oct 13
First implementation task	`whep_probe.py`: one JPEG from a Playground stream via aiortc, with SDPs and HTTP responses logged

Sources

All opened on Oct 11, 2026.

- Hackathon overview, requirements and prizes
- Official rules
- Resources page
- FAQs
- Forum: one project in multiple tracks
- Forum: what the Alexa+ track looks for
- Forum: what the Fire TV track looks for
- Forum: Ring simulator
- Forum: Bedrock quotas at 0 on a new account
- Ring Partner API documentation
- Ring API development guide
- Ring release notes
- Ring computer vision guidelines
- Ring Appstore MCP server
- ring-api-helloworld
- ring-sandbox emulator
- Ring Video Descriptions announcement
- Bee getting started, Bee MCP, Bee realtime

- Vega SDK install requirements
- React Native for Vega accessibility
- Multi-TV React Native sample
- Strands Agents SDK
- Bedrock AgentCore Python SDK
- MCP Apps overview
- EasyPost tracking guide
- Kiro Crew launch coverage
- Prime Video AI and audio description
- 3Play Media on audio description cost
- RNIB study of AI-generated audio description
- ReleaseGuard AI, another Alexa+ entrant
- Merchant help page on delivered-but-missing packages
- VerifyAI proof-of-delivery for 3PLs